



3

11

1320 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

Creating Classes and Objects

د. محمد الأحمد / د. عبد الرحمن اللحام



RB Informatics; 23/04/2025

البرمجة 2

Creating Classes

سنبدأ بمثال بسيط ومن ثم سنبدأ بشرح فكرة إنشاء ال classes:

```
public class Welcome {
    public static void main(String[] args) {
        System.out.println("welcome to Java programming");
    }
}
```

- كل برنامج مكتوب بلغة Java يجب أن يكون فيه class واحد على الأقل (في مثالنا السابق قمنا بإنشاء class اسمه Welcome).
- يمكن أن يبدأ اسم ال class بحرف صغير (small letter) ولكن المتعارف عليه أن يبدأ بحرف كبير (Capital Letter).
- يجب أن يكون اسم ال class مطابقاً لاسم الملف الموجود فيه (أي أنه في مثالنا السابق يجب أن يكون اسم الملف "Welcome.java" وإلا سيكون هناك خطأ في البرنامج).

■ **ملاحظة:** إن إنشاء أكثر من class من النوع public في نفس الملف يعد خطأ (compilation error).

تعريف ال class:

مجموعة من البيانات (data) والطرق (تسمى "methods" وهي مثل التوابيع في لغة C++)، وهذه الطرق والبيانات تعمل كمخطط لإنشاء كائنات (objects) جديدة في ذلك ال class.

٤ إن تعريف ال class يتكون عادة من:

1. Access modifier: تحدد إمكانية وصول باقي ال classes لهذا ال class.
2. "class keyword": عند تعريف ال class يجب كتابة الكلمة المفتاحية "class" قبل اسمه (فهي تشير إلى أن كتلة الأسطر البرمجية التالية تحدد class).
3. Instance Fields: تحتوي على متغيرات وثوابت تستخدم من قبل كائنات الصف.

4. Constructors: عبارة عن methods تمتلك نفس اسم ال class، وتستخدم في تحديد قيم أولية لأي object ينتمي إلى ذلك ال class.
5. Instance methods: تعريف الوظائف (functions) التي يمكن أن تعمل على البيانات في ال class.
6. Class fields: تحتوي المتحولات والثوابت التي تنتمي لل class، وهذه المتحولات والثوابت تكون مشتركة بين جميع ال objects التي تنتمي لنفس ال class.
7. Class methods: تستخدم للتحكم بقيم ال class fields.

↪ مثال:

```
public class Rental {
    //Class variables
    int LateFee;
    // Instance variables
    int rented;
    String rentalDate;
    float rentalAmountDue;
    ...
    // Instance methods
    float getAmountDue (int rentId) {
        ...
    }
    ...
}
```

:Using Java Classes

↪ يوضح العلاقة بين المفاهيم

- الصفوف (Classes) تحتوي على الخصائص (Attributes) والدوال (Methods).
- الكائنات (Objects) تنشأ من الصفوف وتستخدم كمراجع (Object References).
- الحزم (Package) تستخدم لتجميع الصفوف ذات الصلة المنطقية، وتتحكم في الوصول من البرامج الخارجية.

Comparing Classes and Objects:

↪ يقارن بين الصف والكائن

- الكائن (Object): هو حالة (instance) من الصف، ويمتلك ذاكرة خاصة به.
- الصف (Class): هو التعريف أو القالب الذي يجب تحميله لإنشاء الكائنات.

```
public void displayDetails()
    public void setRating()
        private String title;
        private String rating;
```

■ **ملاحظة:** إن ال object هو نموذج عن ال class، وال objects تمتلك ذاكرة خاصة بها.

Creating Objects

إن ال Objects عادة تُنشأ عن طريق استخدام العامل new. ويكون تعريف ال object على الشكل التالي:

```
ClassName objectRef ;
objectRef = new ClassName();
```

وبشكل مختصر:

```
ClassName objectRef = new ClassName ();
```

• (حيث objectRef تعني اسم ال object المراد تعريفه.)

مثال:

```
Movie mov1 = new Movie();
```

استخدام العامل new:

يقوم العامل "new" بالإجراءات الآتية:

1. تخصيص وتهيئة ذاكرة لل objects الجديدة.
2. يستدعي طريقة تهيئة (initialization method) خاصة في ال class تسمى "constructor".

مقارنة بين الأنواع الأساسية (Primitives) والكائنات (Objects):

المتغيرات من النوع البدائي (Primitive Variables) تحمل قيمة، بينما متغيرات الكائنات (Object Variables) تحمل مراجعاً.

مثال:

Primitive Variables

int x = 5; ← تخزين القيمة 5 مباشر في الذاكرة.

Object variables: String s = "Hello"; ← تخزين عنواناً (Reference) يشير إلى مكان ال Object في الذاكرة.

استخدام المرجع null:

- يمكن تعيين قيمة null لمرجع كائن (object references)، ولكن لا يمكن تعيينها لمتغير بدائي (primitive).
- يمكن مقارنة مراجع الكائنات ب null.
- يمكن إزالة الارتباط ب object عن طريق ضبط ال object reference على null.
- التخلص من object:
- عند الانتهاء من استخدام object، يمكن تعيين ال object reference ك null، وهذا يعني أن المتغير لم يعد يشير إلى ال object.
- عندما تنعدم مرجعيات (references) لأي object (أي بعد التعيين ب null)، يصبح ال object غير مستخدماً، وبالتالي يكون جاهزاً للحذف بواسطة ال "Garbage collector".
- آلة جافا الافتراضية (JVM) تقلل تلقائياً من عدد المراجع النشطة إلى كائن عندما يتم إلغاء الإشارة إليه، أو يخرج من نطاقه، أو يتم استخدام المرجع المخزن بمرجع آخر.

Primitive data types: ↩

Primitive type	Size	Wrapper type
boolean	----	Boolean
char	16-bit	Character
Byte	8-bit	Byte
short	16-bit	Short
Int	32-bit	Integer
long	64-bit	Long
float	32-bit	Float
double	64-bit	Double
void	---	Void

↩ لا تغير من بنية جهاز إلى آخر كما يحدث في معظم اللغات.

- جميع الأنواع الرقمية موقعة، لذا لا تبحث عن أنواع غير موقعة.
- boolean data type يأخذ القيمتين الحرفيتين: true / false
- الأنواع البدائية للبيانات (primitive data type) لها أيضاً صفوف مقابلة لها.

↩ إن ال primitive data types تمتلك classes، مثال:

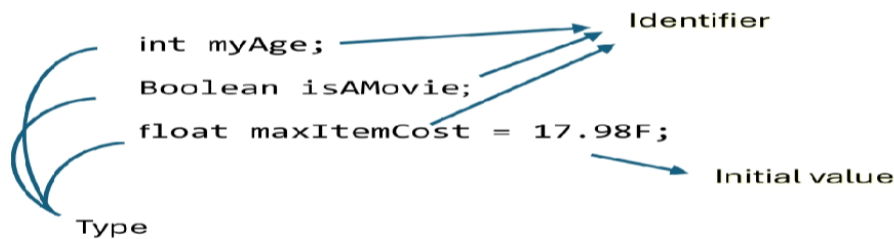
Character C = new character ('x')

ماهي المتغيرات؟

- المتغير هو وحدة أساسية في التخزين.
- يجب تعريف المتغيرات بشكل صريح.
- كل متغير له نوع (type)، معرف (identifier)، ونطاق (scope).
- يصف ثلاثة أنواع من المتغيرات: صف، مثل، وطريقة
- تعيين المراجع: تعيين مرجع إلى مرجع آخر يؤدي إلى وجود مرجعين يشيران إلى نفس الكائن.

```
Movie mov1 = new Movie("Gone...");
mov1
Movie mov2 = mov1;
mov2
title: "Gone with..."
rating: "PG"
```

■ **ملاحظة:** المتغيرات يمكن أن تعطى قيمة افتراضية .. وإذا لم تعط فإنها تعطى القيمة 0.



ما هو الباني constructor؟

لضمان التهيئة (initialization) الصحيحة لل object في java، يجب على ال class أن يوفر constructor. عند إنشاء object جديد من ال class، يتم استدعاء ال constructor تلقائياً. ويتميز بالخصائص التالية:

- يصرح عنه عادة بالكلمة المفتاحية public.
 - يجب أن يحمل نفس اسم ال class تماماً.
 - لا يحدد له نوع إرجاع (return type)، حتى void غير مسموحة.
- يقوم المترجم بإنشاء constructor افتراضي بدون معاملات (باراميترات) تلقائياً ويسمى "No-Arg Constructor"، ولكن فقط إذا لم نقوم بتعريف أي constructor في ال class. أما إذا قمنا بتعريف أي constructor صراحة – سواء كان يحتوي على معاملات أم لا – فإن المترجم لا يقوم بإنشاء ال constructor الافتراضي.

تعريف وزيادة تحميل (overloading) ال (Constructors):

لأن كل constructor يجب أن يمتلك نفس اسم ال class، فإن التمييز بين ال constructors المختلفة ضمن نفس ال class يتم عن طريق عدد المعاملات (arguments) أو نوع المعاملات.

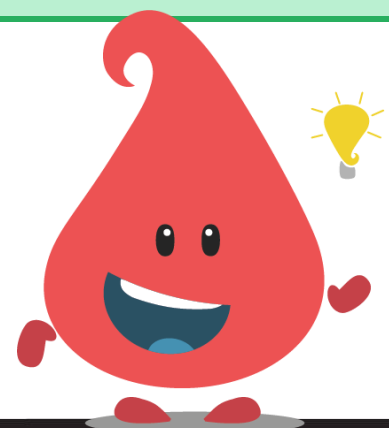
مثال:

```
public class Movie {
    private String title;
    private String rating = "PG";
    public Movie () {
        title = "Last Action ...";
    }
    public Movie(String newTitle) {
        title = newTitle;
    }
}
```

مشاركة الكود بين ال constructors:

يمكن أي constructor استدعاء أي constructor آخر باستخدام "this()"

```
public class Movie {
    private String title;
    private String rating;
    public Movie () {
        this("G"); }
    public Movie(String newRating) {
        rating = newRating;
    }
}
```



عند استخدام الصيغة this() لاستدعاء باني آخر داخل نفس الكلاس، يجب الانتباه إلى بعض القواعد الأساسية:

1. يجب أن تكون this() هي أول سطر في ال constructor، لا يمكن كتابة أي تعليمات قبلها، وإلا سيحدث خطأ في الترجمة.
2. يجب أن تتطابق المعاملات (Arguments) التي تمرر إلى this() مع المعاملات التي يتطلبها ال constructor المُستدعى (أي نفس الترتيب و الأنواع).

تهيئة المتغيرات:

- لا تتم بتهيئة متغيرات ال class في ال constructor، فال constructors تستخدم لتهيئة متغيرات الكائن (instance variables)، وليس متغيرات ال class.
- يمكن تهيئة متغيرات ال class عند التصريح عنها.
- جميع متغيرات ال class تهيأ تلقائياً بقيم افتراضية حسب نوع البيانات.

تهيئة متغيرات ال class:

التهيئة المعقدة لمتغيرات الفصل:

تنفذ التهيئة المعقدة لمتغيرات الفصل داخل كتلة تهيئة ثابتة (Static initializ).

خصائص ال static initializer:

- ليس لها اسم.
- لا تعيد أي قيمة.
- تبدأ ب "static"
- يليها كتلة من الأقواس بين أقواس {}.
- تشبه ال constructor لكنها تنفذ مرة واحدة فقط ولا تعتمد على أي object من ال class.

مثال توضيحي:

```
public class Movie {
    private static double minPrice;
    static {
        Date todayDate = new Date();
        minPrice = getMinPrice (todayDate);
    }
}
```

```
class Spoon {
    static int i;
    static{
        i = 47;
    }
}
```

// هذا الكود مثل أي (static initialization) ينفذ مرة واحدة فقط

// إما عندما قمت بإنشاء أول object من ال class

// أو عند الوصول إلى أول عضو ثابت (static) في ال class حتى لو لم ينشأ أي object

تمرير الأنواع الأساسية (Primitives) ضمن الطرق (Methods):

➤ عند تمرير قيمة primitive أو object reference ضمن method تتولد نسخة من القيمة.

```
public void aMethod(int arg){
    if (arg < 0 || arg > 100)
        arg = 0;
    System.out.println("arg: " + arg);
}
int num = 150;
anObj.aMethod(num);
System.out.println("num: " + num);
```

➤ الخرج:

arg: 0

num: 150

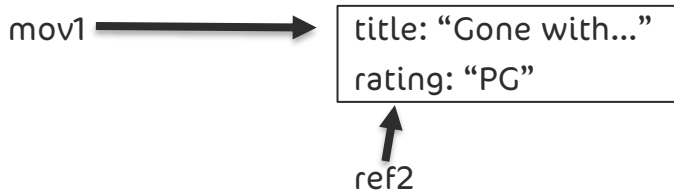


تمرير object references ضمن Methods:

عندما نمرر object reference ضمن method فإن ال object لا يُنسخ وإنما يُنسخ المؤشر (pointer) لل object.

مثال:

```
Movie mov1 = new
    Movie("Gone...");
mov1.setRating("PG");
public void aMethod(Movie ref2){
    ref2.setRating("R");
}
```



الجزء الأول: صفوف وكائنات

```
class Tag {
    Tag(int marker) { // "Tag" مع marker قيمة يطبع باني
        System.out.println("Tag(" + marker + ")");
    }
}

class Card {
    Tag t1 = new Tag(1); // Tag(1) بـ t1 تهينة
    Card() { // "Card" يطبع باني Card
        System.out.println("Card()");
        t3 = new Tag(33);
    }
    Tag t2 = new Tag(2); // Tag(2) بـ t2 تهينة
    void f() { // "f()" تطبع دالة
        System.out.println("f()");
    }
    Tag t3 = new Tag(3); // Tag(3) بـ t3 تهينة
}
```

```
public class BBM {
    public static void main(String[] args) { // البرنامج تبدأ رئيسية دالة
        Card t = new Card(); // كائن إنشاء Card
        t.f(); // دالة استدعاء f }}

```

الجزء الثاني: تمرير القيم

```
class BBB {
    BBB(int marker) { // قيمة مع "BBB" يطبع باني
        System.out.println("BBB(" + marker + ")");
    }
    void f(int marker) { // قيمة مع "f" تطبع دالة
        System.out.println("f(" + marker + ")");
    }
}

class aaa {
    BBB b1 = new BBB(1); //تهيئة b1
    void f2() { // b2.f(1) وتستدعي "aaa" تطبع دالة
        System.out.println("aaa()");
        b2.f(1);
    }
    void f2(int marker) { // قيمة مع "f2" تطبع دالة
        System.out.println("f2(" + marker + ")");
    }
    static BBB b2 = new BBB(2); // ثابت كمتغير BBB(2) بـ b2 تهيئة
}

class ccc {
    BBB b3 = new BBB(3); //تهيئة b3
    ccc() { // b4.f(2) ويستدعي "ccc" يطبع باني
        System.out.println("ccc()");
        b4.f(2);
    }
    BBB b4 = new BBB(4); //تهيئة b4
    void f3(int marker) { // قيمة مع "f3" تطبع دالة
        System.out.println("f3(" + marker + ")");
    }
}

```

```

static BBB b5 = new BBB(5); // ثابت كمتغير b5 بـ BBB(5) تهيئة
}

public class BM {
    public static void main(String[] args) { // البرنامج تبدأ رئيسية دالة
        System.out.println("Creating new ccc() in main"); // رسالة طباعة
        new ccc(); // كائن إنشاء
        System.out.println("Creating new ccc() in main"); // رسالة طباعة
        new ccc(); // آخر كائن إنشاء
        t2.f2(1); // لت2 استدعاء f2(1)
        t3.f3(1); // لت3 استدعاء f3(1)
    }
    static aaa t2 = new aaa(); // ثابت كمتغير t2 بـ aaa تهيئة
    static ccc t3 = new ccc(); // ثابت كمتغير t3 بـ ccc تهيئة
}

```

🔚 انتهت المحاضرة

